Where Is My Doctor?

Database Schema, ER Model & API Specification

V1 Technical Design • September 2026

This document defines the proposed V1 PostgreSQL/PostGIS domain model, relationships, indexes, search-oriented data structures, API boundaries and representative REST contracts. It is intended to be the implementation baseline before coding the backend.

1. Database Design Principles

- PostgreSQL is the source of truth for healthcare, location, schedule and report data.
- PostGIS stores practice and location coordinates and performs geographic filtering.
- Doctor and establishment are separate entities because one doctor can practice at many establishments and one establishment can host many doctors.
- Schedules belong to a doctor-practice relationship, not directly to a doctor.
- Recurring schedules and temporary availability overrides are separate concepts.
- Provider data is admin-managed in V1.
- Community reports never directly mutate healthcare data; they enter an admin review workflow.
- Patient discovery does not require authentication.
- Sensitive patient health history is intentionally outside V1.

2. Core Entity Relationship

```
User
 ├── MyVisit
 └── Report

Doctor
 └── DoctorPractice
      ├── Establishment
      │     └── PracticeLocation
      ├── Schedule
      └── AvailabilityOverride

Doctor
 ├── DoctorSpecialty ── Specialty
 └── DoctorLanguage

HealthTopic
 ├── HealthTopicAlias
 └── HealthTopicSpecialty ── Specialty

Location
 └── parent Location

Report
 ├── Doctor (optional)
 ├── Establishment (optional)
 └── PracticeLocation (optional)
```

3. PostgreSQL Tables

3.1 users

```
id UUID PK
name VARCHAR(150)
phone VARCHAR(20) UNIQUE
email VARCHAR(255) UNIQUE NULL
password_hash TEXT NULL
preferred_language VARCHAR(10) DEFAULT 'en'
role VARCHAR(30) DEFAULT 'patient'
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

Admin roles can later be split into super_admin, admin, reviewer, content_editor, etc.

3.2 doctors

```
id UUID PK
name VARCHAR(200)
display_name VARCHAR(200)
photo_url TEXT NULL
bio TEXT NULL
qualifications TEXT
experience_years SMALLINT NULL
consultation_fee NUMERIC(10,2) NULL
phone VARCHAR(20) NULL
registration_number VARCHAR(100) NULL
registration_council VARCHAR(150) NULL
verification_status VARCHAR(30)
verified_at TIMESTAMPTZ NULL
last_verified_at TIMESTAMPTZ NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

3.3 specialties

```
id UUID PK
name VARCHAR(150)
slug VARCHAR(150) UNIQUE
description TEXT NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

3.4 doctor_specialties

```
doctor_id UUID FK doctors(id)
specialty_id UUID FK specialties(id)
is_primary BOOLEAN DEFAULT false
PRIMARY KEY (doctor_id, specialty_id)
```

3.5 doctor_languages

```
doctor_id UUID FK doctors(id)
language_code VARCHAR(10)
PRIMARY KEY (doctor_id, language_code)
```

3.6 establishments

```
id UUID PK
name VARCHAR(250)
```

```
slug VARCHAR(250) UNIQUE
type VARCHAR(40)
description TEXT NULL
phone VARCHAR(20) NULL
email VARCHAR(255) NULL
website_url TEXT NULL
opening_hours JSONB NULL
is_verified BOOLEAN DEFAULT false
last_verified_at TIMESTAMPTZ NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

type values: hospital, clinic, nursing_home, pharmacy, doctor_chamber, diagnostic_center, other.

3.7 practice_locations

```
id UUID PK
establishment_id UUID FK establishments(id)
address_line TEXT
locality VARCHAR(150) NULL
pincode VARCHAR(10) NULL
location_id UUID FK locations(id) NULL
latitude DOUBLE PRECISION
longitude DOUBLE PRECISION
geo GEOGRAPHY(Point, 4326)
phone VARCHAR(20) NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

The geo column should be generated/populated from longitude and latitude and indexed with GiST.

3.8 doctor_practices

```
id UUID PK
doctor_id UUID FK doctors(id)
practice_location_id UUID FK practice_locations(id)
consultation_fee NUMERIC(10,2) NULL
notes TEXT NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
UNIQUE (doctor_id, practice_location_id)
```

3.9 schedules

```
id UUID PK
doctor_practice_id UUID FK doctor_practices(id)
day_of_week SMALLINT
start_time TIME
end_time TIME
effective_from DATE NULL
effective_until DATE NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

day_of_week should use ISO numbering: Monday=1 through Sunday=7.

3.10 availability_overrides

```
id UUID PK
doctor_practice_id UUID FK doctor_practices(id)
date DATE
status VARCHAR(40)
start_time TIME NULL
end_time TIME NULL
reason TEXT NULL
created_by UUID FK users(id)
created_at TIMESTAMPTZ
```

Examples: unavailable, confirmed_available, modified_hours.

3.11 locations

```
id UUID PK
name VARCHAR(200)
type VARCHAR(40)
parent_id UUID FK locations(id) NULL
state_code VARCHAR(20) NULL
district_code VARCHAR(50) NULL
pincode VARCHAR(10) NULL
latitude DOUBLE PRECISION NULL
longitude DOUBLE PRECISION NULL
geo GEOGRAPHY(Point,4326) NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

This supports State → District → Subdivision/Block → Town/Village/Locality without forcing that hierarchy on the user interface.

3.12 health_topics

```
id UUID PK
name VARCHAR(150)
slug VARCHAR(150) UNIQUE
category VARCHAR(100)
description TEXT NULL
is_active BOOLEAN DEFAULT true
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

3.13 health_topic_aliases

```
id UUID PK
topic_id UUID FK health_topics(id)
language_code VARCHAR(10)
phrase VARCHAR(200)
normalized_phrase VARCHAR(200)
UNIQUE (language_code, normalized_phrase)
```

3.14 health_topic_specialties

```
topic_id UUID FK health_topics(id)
specialty_id UUID FK specialties(id)
priority SMALLINT DEFAULT 0
PRIMARY KEY (topic_id, specialty_id)
```

3.15 my_visits

```
id UUID PK
user_id UUID FK users(id)
```

```
doctor_id UUID FK doctors(id)
practice_location_id UUID FK practice_locations(id)
visit_date DATE
visit_time TIME NULL
notes TEXT NULL
reminder_enabled BOOLEAN DEFAULT true
reminder_sent_at TIMESTAMPTZ NULL
status VARCHAR(30) DEFAULT 'planned'
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

MyVisit is a reminder/planning feature, not an appointment or booking record.

3.16 reports

```
id UUID PK
reporter_user_id UUID FK users(id) NULL
doctor_id UUID FK doctors(id) NULL
establishment_id UUID FK establishments(id) NULL
practice_location_id UUID FK practice_locations(id) NULL
report_type VARCHAR(50)
message TEXT NULL
reported_date DATE NULL
status VARCHAR(30) DEFAULT 'pending'
admin_notes TEXT NULL
reviewed_by UUID FK users(id) NULL
reviewed_at TIMESTAMPTZ NULL
resolution TEXT NULL
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

4. Important Constraints

- A doctor_practice must reference an active doctor and active practice location.
- A schedule must have start_time < end_time.
- effective_from must not be after effective_until.
- Latitude must be between -90 and 90; longitude between -180 and 180.
- A primary specialty should be unique per doctor.
- A report must identify at least one relevant target: doctor, establishment or practice location.
- A MyVisit must reference a doctor-practice-compatible location.
- Deleted healthcare records should generally be soft-deactivated rather than physically deleted because historical reports and auditability matter.

5. Recommended Indexes

```
-- Doctor / establishment text
CREATE INDEX doctors_name_trgm_idx
ON doctors USING gin (name gin_trgm_ops);

CREATE INDEX establishments_name_trgm_idx
ON establishments USING gin (name gin_trgm_ops);

-- Geographic
CREATE INDEX practice_locations_geo_idx
ON practice_locations USING gist (geo);

CREATE INDEX locations_geo_idx
```

```
ON locations USING gist (geo);

-- Relationships
CREATE INDEX doctor_practices_doctor_idx
ON doctor_practices (doctor_id);

CREATE INDEX doctor_practices_location_idx
ON doctor_practices (practice_location_id);

CREATE INDEX schedules_practice_day_idx
ON schedules (doctor_practice_id, day_of_week);

-- Health search
CREATE INDEX health_alias_normalized_idx
ON health_topic_aliases (language_code, normalized_phrase);
```

6. Search Request Contract

```
GET /api/v1/search/doctors

Query parameters:
q
latitude
longitude
radiusKm
locationId
date
time
specialty
healthTopic
language
availableOnly
page
limit
```

The client can send either coordinates or a selected locationId. Coordinates are preferred for nearby/radius search.

7. Search Processing

```
Request
  ↓
Validate query
  ↓
Resolve location
  ↓
Normalize text
  ↓
Detect intent
  ↓
Resolve doctor / establishment / specialty / topic
  ↓
Build candidate query
  ↓
PostGIS radius filter
  ↓
Schedule filter
  ↓
Availability override
  ↓
```

```
Text relevance
  ↓
Distance relevance
  ↓
Freshness
  ↓
Pagination
  ↓
Response
```

8. Example Search SQL Strategy

```
SELECT
    d.id,
    d.name,
    s.name AS specialty,
    e.name AS establishment,
    pl.address_line,
    ST_Distance(
        pl.geo,
        ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography
    ) AS distance_m
FROM doctors d
JOIN doctor_specialties ds ON ds.doctor_id = d.id
JOIN specialties s ON s.id = ds.specialty_id
JOIN doctor_practices dp ON dp.doctor_id = d.id
JOIN practice_locations pl ON pl.id = dp.practice_location_id
JOIN establishments e ON e.id = pl.establishment_id
WHERE d.is_active = true
  AND dp.is_active = true
  AND pl.is_active = true
  AND s.slug = :specialty
  AND ST_DWithin(
      pl.geo,
      ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography,
      :radiusMeters
  )
ORDER BY distance_m ASC
LIMIT :limit OFFSET :offset;
```

The production query will additionally join/filter schedules, overrides, taxonomy matches and ranking signals.

9. Relevance Score

```
score =
    exact_name_match
  + specialty_match
  + topic_match
  + date_match
  + availability_match
  + distance_score
  + freshness_score
  + establishment_text_match
```

The score is search relevance only. It must not be presented as a doctor quality, medical skill, popularity or safety score.

10. Radius Expansion

```
Initial radius
    ↓
5 km
    ↓ no results
10 km
    ↓ no results
25 km
    ↓ no results
50 km
```

Expansion should stop when useful candidates are found or when a configured maximum is reached. The response should identify the effective search area so users understand why a distant result was shown.

11. Public API Structure

Method	Endpoint	Purpose
GET	/api/v1/search/doctors	Doctor discovery/search
GET	/api/v1/doctors/:id	Doctor profile
GET	/api/v1/doctors/:id/practices	Doctor practice locations
GET	/api/v1/establishments/:id	Establishment profile
GET	/api/v1/establishments/:id/doctors	Doctors practicing there
GET	/api/v1/locations/search	Location suggestions
GET	/api/v1/specialties	Specialty list
GET	/api/v1/health-topics	Health topic list
POST	/api/v1/my-visits	Create planned visit
GET	/api/v1/my-visits	List planned visits
PATCH	/api/v1/my-visits/:id	Update planned visit
DELETE	/api/v1/my-visits/:id	Cancel planned reminder
POST	/api/v1/reports	Submit report
GET	/api/v1/me	Current user
POST	/api/v1/auth/register	Patient registration
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/admin/doctors	Create doctor
PATCH	/api/v1/admin/doctors/:id	Update doctor
POST	/api/v1/admin/establishments	Create establishment
PATCH	/api/v1/admin/establishments/:id	Update establishment
POST	/api/v1/admin/practice-locations	Create practice location
POST	/api/v1/admin/doctors/:id/practices	Link doctor to practice
POST	/api/v1/admin/doctor-practices/:id/schedules	Create schedule
PATCH	/api/v1/admin/schedules/:id	Update schedule
POST	/api/v1/admin/availability-overrides	Create temporary availability override
GET	/api/v1/admin/reports	List/filter reports
PATCH	/api/v1/admin/reports/:id	Review/resolve/reject report
POST	/api/v1/admin/health-topics	Create health topic

POST	/api/v1/admin/health-topics/:id/aliases	Add search alias

12. Admin API Structure

13. Example Search Response

```
{
  "data": [
    {
      "doctor": {
        "id": "...",
        "name": "Dr. Amit Das",
        "specialties": ["Dermatology"]
      },
      "practice": {
        "id": "...",
        "establishment": "Maa Medical Store",
        "distanceKm": 1.8,
        "address": "..."
      },
      "schedule": {
        "day": "Thursday",
        "start": "17:00",
        "end": "20:00"
      },
      "availability": "scheduled",
      "verification": {
        "lastVerifiedAt": "2026-09-12"
      }
    }
  ],
  "meta": {
    "radiusKm": 10,
    "total": 1
  }
}
```

14. Authentication Boundaries

- Public endpoints: search, doctor profiles, establishment profiles, location suggestions, specialties and health topics.
- Authenticated patient endpoints: My Visits, saved preferences and report submission where required.
- Admin endpoints require authenticated admin roles.
- Never expose password hashes, internal admin notes or private audit information through patient APIs.
- Use access tokens with short expiry and refresh tokens with revocation/rotation strategy.

15. Data Freshness

Healthcare records should include verification metadata.

```
last_verified_at
verified_by
verification_status
updated_at
```

Search results can use freshness as a relevance signal, while the UI can show controlled warnings such as "Schedule may have changed — call to confirm."

16. Report Workflow

```
Patient Report
     ↓
PENDING
     ↓
Admin Review
   ┌──┴──────────┐
   ↓             ↓
VERIFY       REJECT
  ↓
Update data
  ↓
RESOLVED
```

Multiple reports should be treated as a signal for verification, not as an automatic data update.

17. Auditability

Because administrators initially control the directory, production should include an audit log for important changes.

```
audit_logs
----------
id
actor_user_id
entity_type
entity_id
action
old_values JSONB
new_values JSONB
created_at
```

This makes schedule and profile changes traceable.

18. V1 Database Scope Exclusions

- Appointment booking tables
- Payment tables
- Prescription records
- Medical records
- Telemedicine sessions
- Doctor self-service account tables
- Complex patient medical history

19. Implementation Order

- Create PostgreSQL database and enable PostGIS/pg_trgm.
- Create locations and geographic indexes.
- Create doctor, specialty, establishment and practice tables.
- Create doctor-practice schedules and availability overrides.
- Create healthcare taxonomy and aliases.
- Implement admin CRUD APIs.
- Implement location search.
- Implement deterministic doctor search.
- Add PostGIS radius filtering.
- Add schedule/date filtering.
- Add relevance ranking.
- Add patient profiles/My Visits/reports.
- Add authentication and role protection.
- Add API documentation and integration tests.
- Load initial real-world data and validate search behavior.

20. Technical Decision Summary

```
Database: PostgreSQL
Geospatial: PostGIS
Fuzzy text: pg_trgm
Search text: PostgreSQL FTS
ORM: Drizzle
Backend: Express + TypeScript
Patient: React Native + Expo + TypeScript
Admin: React + Vite + TypeScript
Validation: Zod
Server state: TanStack Query
Auth: JWT + refresh tokens
Localization: i18next
Notifications: Expo Notifications
Deployment: Linux VPS + Nginx + Docker

Core search:
User language
 → canonical healthcare concept
 → geographic origin
 → candidate practices
 → schedule/availability
 → relevance ranking
 → doctor results
```

21. Next Document

The next implementation artifact should define the API contracts in greater depth and provide the Drizzle ORM schema/migration structure, including TypeScript types, enums, relations, indexes and representative search-service pseudocode.